Clemens Höglinger

Temporally Robust Detection and State Modelling of Cats and Prey Using Deep Learning

Bachelor Thesis

to obtain the academic degree of

Bachelor of Science in Engineering (BSc)

Bachelor's degree program
Software Engineering
University of Applied Sciences Upper Austria
Hagenberg

Year of submission: 2026

Supervision: Dr. Gerald Adam Zwettler

© Copyright 2026 Clemens Höglinger

This work is published under the conditions of the Creative Commons License *Attribution-NonCommercial-NoDerivatives 4.0 International* (CC BY-NC-ND 4.0)—see https://creativecommons.org/licenses/by-nc-nd/4.0/.

Affidavit

I hereby declare that I have written the presented thesis independently and have used only the references and aids indicated. All passages taken verbatim or paraphrased from the references are clearly identified as such. This thesis has not been submitted, in whole or in part, in the same or a similar form, to any other examination authority, and is identical to the electronically submitted text document.

Hagenberg, August 20, 2026

Clemens Höglinger

Documentation of AI Tools Usage

Regarding the use of generative artificial intelligence tools such as chatbots, image generators, programming applications and paraphrasing or translation tools, I hereby document that I used generative artificial intelligence tools, to

- search for literature.
- proofread parts of the thesis and improve the linguistic style.

Contents

Affidavit ii

Documentation of AI Tools Usage iii

Kurzfassung v

Abstract vi

1 Introduction 1

2 Theoretical Background 2
2.1 Object Detection with YOLO . . . 2
2.2 Multi-Object Tracking with ByteTrack . . . 2
2.3 Image Segmentation with YOLO . . . 3
2.4 State Machines . . . 3

3 Materials and Experimental Basis 5
3.1 Datasets . . . 5
3.2 Hardware and Software Environment . . . 5

4 Methodology and Development 7
4.1 Overall Architecture and Data Flow . . . 7
4.2 Preprocessing for Infrared Night Vision . . . 8
4.3 Training and Adaptation of the Object Detector (YOLO26n) . . . 8
4.4 Automated Generation of the Segmentation Dataset . . . 8
4.5 Crop Extraction . . . 9
4.6 Training of the Segmentation Model (YOLO Segmentation) . . . 9
4.7 Object Tracking and Development of the State Model . . . 9

5 Design and Implementation 11
5.1 Software Architecture and Core Components . . . 11
5.1.1 The Module `models.py`: The `CatFlapPipeline` . . . 11
5.1.2 The Module `state_machine.py`: Temporal Aggregation . . . 12
5.1.3 The Orchestration (`processor.py`) . . . 13
5.2 The Connection to the Hardware (`catflap.py`) . . . 13
5.3 Execution Environments . . . 13
5.3.1 The Local Test Environment (`main.py`) . . . 13

5.3.2 The Test Environment on the Raspberry Pi (`pi_main.py`) 14
5.3.3 The Real World (`pi_server.py`) 14

6 Tests and Analyses **15**
6.1 Model Performance (Training & Validation) 15
6.1.1 Performance of Object Detection 15
6.1.2 Performance of Image Segmentation 18
6.2 Evaluation Methodology and Dataset 21
6.2.1 Test Dataset 21
6.2.2 Annotation of the Reference Data 21
6.2.3 Definition of the System Metrics 21
6.3 Evaluation Results 22
6.4 Ablation Studies 23
6.4.1 State Machine 23
6.4.2 Detection and Segmentation versus Detection 26
6.4.3 Greyscale versus Colour Images 28

7 Discussion **29**
7.1 Answering the Central Research Question 29
7.2 Assessment of the Architectural Approach 29
7.2.1 Strengths of the Pipeline Cascading 30
7.2.2 The Role of the Asymmetric Padding 30
7.3 Weaknesses, Limitations and Sources of Error 30
7.4 Local Data Processing 31

8 Summary and Outlook **32**
8.1 Summary of the Findings 32
8.2 Outlook and Possibilities for Further Development 33
8.2.1 Continuous Collection of Hard Negative Examples 33
8.2.2 Loss of Depth Information 33
8.2.3 Hardware Acceleration 33

References **34**
Literature 34

Kurzfassung

Freigängerkatzen bringen regelmäßig Beutetiere wie Mäuse oder Vögel in den Wohnraum. Herkömmliche Katzenklappen können dies nicht verhindern, da sie lediglich zwischen bekannten und fremden Tieren unterscheiden. Eine kamerabasierte Beuteerkennung ist schwierig, da das Beutetier im Bild klein und oft teilweise verdeckt ist und die Vorhersagen eines Modells durch Bewegungsunschärfe und wechselnde Lichtverhältnisse von Einzelbild zu Einzelbild schwanken. Entscheidungen auf Basis einzelner Bilder führen daher zu einem ständigen Wechsel zwischen Sperren und Freigeben der Klappe.

Die vorliegende Arbeit untersucht, ob eine Verarbeitungskette aus Objekterkennung, Objektverfolgung, Bildsegmentierung und zeitlicher Zustandsmodellierung eine robustere Beuteerkennung ermöglicht als die Auswertung einzelner Bilder. Ein feinabgestimmter YOLO-Objektdetektor lokalisiert die Gesichter der Katzen, ByteTrack ordnet die Erkennungen über die Zeit eindeutigen Identitäten zu, und ein YOLO-Segmentierungsmodell bewertet einen nach unten erweiterten Ausschnitt jedes Katzengesichts auf das Vorhandensein von Beute. Ein Zustandsautomat aggregiert die Konfidenzwerte jeder Katze über ein gleitendes Zeitfenster und entscheidet erst dann über die Freigabe der Klappe. Für den Nachtbetrieb arbeitet das System auf Graustufenbildern. Die Trainingsdaten des Segmentierungsmodells wurden mangels realer Aufnahmen mithilfe generativer KI erzeugt, und die Modelle wurden für den Betrieb auf einem Raspberry Pi 5 optimiert.

Die Evaluierung anhand von 20 KI-generierten Videosequenzen zeigt, dass das Gesamtsystem nur in 1,0 % der Einzelbilder mit Beute die Klappe fälschlicherweise freigibt und keine saubere Katze aussperrt. In Ablationsstudien senkt der Zustandsautomat den Sicherheitsfehler gegenüber der Einzelbildentscheidung von 17,0 % auf 1,0 % und den Komfortfehler von 19,7 % auf 3,3 %. Der zweistufige Ansatz sperrt saubere Katzen deutlich seltener als ein einstufiger Detektor (3,3 % gegenüber 29,7 %).

Die Ergebnisse bestätigen, dass die Kombination aus Objekterkennung, Objektverfolgung und zeitlicher Zustandsmodellierung deutlich stabilere Entscheidungen liefert als eine Einzelbildauswertung und sich auf kostengünstiger, lokaler Hardware umsetzen lässt.

Abstract

Outdoor cats regularly bring prey such as mice or birds into the home. Conventional cat flaps cannot prevent this, as they only distinguish between known and unknown animals. Camera-based prey detection is challenging because the prey is small and often partially occluded, and the predictions of a model fluctuate from frame to frame due to motion blur and changing lighting conditions. Decisions based on individual frames therefore lead to the flap constantly switching between locking and unlocking.

This thesis investigates whether a processing pipeline consisting of object detection, object tracking, image segmentation and temporal state modelling enables more robust prey detection than the evaluation of individual frames. A fine-tuned YOLO object detector localises the faces of the cats, ByteTrack assigns the detections to consistent identities over time, and a YOLO segmentation model examines a downward-extended crop of each cat's face for the presence of prey. A state machine aggregates the confidence scores of each cat over a sliding time window and only then decides whether to unlock the flap. To support operation at night, the system works on grayscale images. Due to the lack of real recordings, the training data for the segmentation model was created using generative AI, and the models were optimised for operation on a Raspberry Pi 5.

The evaluation on 20 AI-generated video sequences shows that the overall system wrongly unlocks the flap in only 1.0 % of the frames with prey and never locks out a clean cat. In ablation studies, the state machine reduces the safety error compared to frame-by-frame decisions from 17.0 % to 1.0 % and the comfort error from 19.7 % to 3.3 %. The two-stage approach locks out clean cats considerably less often than a single-stage detector (3.3 % compared to 29.7 %).

The results confirm that the combination of object detection, object tracking and temporal state modelling leads to considerably more stable decisions than frame-by-frame evaluation and can be implemented on low-cost local hardware.

Chapter 1

Introduction

Outdoor cats occasionally bring prey animals, such as mice or birds, into the house, which can be very unpleasant for the owner. These prey animals are often difficult to recognise in camera footage because they are small or occluded. When one tries to solve this problem with pure object detection, individual frames alone often prove to be error-prone [0]. Motion blur or occlusions mean that the prey cannot be recognised with the required accuracy. A snapshot is not sufficient here, which is why an analysis across several frames is required.

This bachelor thesis deals with the detection of prey animals in a cat's mouth supported by artificial intelligence. The goal of this project is to offer an alternative to conventional cat flaps and to spare the cat owner the annoying task of catching prey in the living space.

Since image recognition is, among other things, the centrepiece of this project, I have worked intensively with various algorithms. In the first step, the object detection model YOLO26 [0] is used. YOLO26 achieves a high accuracy while also remaining real-time capable on edge devices. Subsequently, the ByteTrack [0] algorithm is used. This algorithm tracks the detected bounding boxes over time and ensures that the system does not lose sight of the cat, even during brief occlusions. ByteTrack also ensures that each cat is uniquely identified should several cats be in the video frame. The detected regions are then evaluated by a further artificial intelligence (YOLO segmentation), which segments the prey in the image and derives a confidence score for prey from this. To increase reliability and to avoid false alarms, a decision is not made for every video frame; instead, data are collected over several video frames and a decision is made for these data as a whole. For this purpose, a custom state machine was integrated. It evaluates the history of the frames over a sliding time window and then decides whether or not the cat is carrying prey in its mouth. This thesis evaluates to what extent this combination of object detection, segmentation, object tracking and a state machine is a better solution than evaluation based purely on individual frames.

Chapter 2

Theoretical Background

This chapter explains the theoretical background, which forms the foundation of this thesis. Only technologies and algorithms that are used in the final work are explained. These include object detection, object tracking, image segmentation and state machines.

2.1 Object Detection with YOLO

Automatic image analysis can fundamentally be divided into different problem classes [0]. While pure image classification merely answers the question of whether a certain object is present in an image, object detection goes one step further [0]. It determines not only the class of the object but also the exact position at which the object is located.

In this thesis, YOLO [0] in version 26 is used [0]. In contrast to two-stage detectors, which first extract interesting image regions and only classify them in the second step [0], YOLO is a single-stage detector that analyses the entire image in one pass using a convolutional neural network [0] [0]. The image is divided into a grid, and for each grid cell the network predicts several bounding boxes and the associated class probabilities and confidence scores [0]. The bounding box is the frame in which an object is located. The class probability indicates how likely it is that the detected object belongs to the class cat. The confidence score, on the other hand, indicates how certain the network is that there is an object at all [0].

YOLO is indispensable for this thesis, since such a system requires low inference times [0]. On the one hand, this is important so that the system remains reliable and the cat does not have to wait in front of the flap for too long. For each detected cat, the model delivers the exact coordinates of its face, which forms the basis for object tracking.

2.2 Multi-Object Tracking with ByteTrack

An object detection model such as YOLO on its own evaluates each video frame in isolation. If a cat is in a video, each frame of the video is considered separately. Every detected cat is therefore a new cat. Object tracking makes it possible to uniquely assign a cat across an entire video. Multi-object tracking is then the extension to several cats per video [0]. This is absolutely necessary in order to be able to carry out a temporal

analysis of a specific cat.

In this thesis, an object detector (in this case YOLO26) produces the bounding boxes, which contain the exact coordinates of the objects, in our case of the cat's face. In the second step, the tracker takes over. It tries to assign the new detections to the known ones; if this is not possible, a new identity is created [0]. In the case of ByteTrack [0], this assignment is done using motion predictions made by the Kalman filter [0].

ByteTrack differs from classic trackers [0] in that it also uses detections with a low confidence score in order to prevent identity switches. So that this does not lead to false alarms, it first assigns the bounding boxes with high confidence scores [0]. Subsequently, the objects with low confidence scores that could not be assigned in the previous step are compared. This makes the tracking more robust against occlusions [0]. ByteTrack is therefore ideally suited to unpredictable movements and occlusions in the use case of a cat flap.

2.3 Image Segmentation with YOLO

Like object detection, segmentation works on the principle of convolutional neural networks [0]. Modern YOLO versions, such as YOLO26 in our case, offer not only object detection but also segmentation [0]. Owing to their architecture, these networks are designed for fast inference at high accuracy [0].

While object detection shows us where a cat is located, we still do not know whether or not a cat is carrying prey in its mouth. This is where image segmentation comes into play [0]. This approach has the advantage that the segmentation no longer has to understand the whole image; the model only receives a crop around the cat's face. This approach requires a considerably smaller dataset, since the segmentation always receives a similar task with little variance.

The task of the segmentation model is thus to mark the prey in the cat's mouth with pixel accuracy. For each prey animal found, the model delivers a mask and a confidence score. The highest confidence score of a crop indicates how likely it is that the cat is carrying prey. If the model finds no prey, the value is 0.

2.4 State Machines

While the object detector and the segmentation deliver snapshots, which can certainly fluctuate, a state machine helps us with the decision. It aggregates the results of the detector and the segmentation over time, so that the result is not strongly influenced by outliers and becomes more robust against errors [0].

A finite state machine is a model that contains a finite set of states [0]. At any point in time, the model is in exactly one state. The current state of the automaton is determined by certain events or the fulfilment of certain conditions, e.g. if the average confidence score of the last 20 frames is above 0.3, the state machine switches to access denied [0]. A transition from one state usually also triggers actions; in our case, we would open or close the cat flap [0].

A simple example of a state machine is a traffic light; the model represents whether the traffic light is red, yellow or green. The condition for switching is a timer. The action

would be the switching of the traffic light.

In our context, the state machine serves to smooth out inaccuracies of the object detection and segmentation. Instead of immediately changing the state of the cat flap in the case of an outlier, the state transition is tied to conditions (e.g. a threshold over a time window).

Chapter 3

Materials and Experimental Basis

This chapter describes the specific resources and technologies that were used for training the models and for evaluating the pipeline.

3.1 Datasets

Different datasets were required for object detection and image segmentation, since each solves a specific problem.

Dataset for object detection (cat faces):
For training the object detection with YOLO26, a public Kaggle dataset was used ("Cat Dataset" by Crawford [0]). In this dataset, the location of each cat's face in the image is already marked, and it also contains a large variety of cats in a wide range of lighting conditions, poses and backgrounds. The data only had to be modified slightly so that it was suitable for YOLO training.

Dataset for segmentation (prey animals):
In contrast to object detection, where a ready-made dataset existed, I found hardly any data for this problem. Datasets were either too small or of too low quality. To solve this problem, the training data were generated with the help of AI. This made it possible to produce specific scenarios, that is, different prey animals and poses. Subsequently, the prey animals were annotated as segmentation masks using the AI-assisted annotation of Roboflow [**roboflow2025**] and checked manually. In this way, the YOLO26 model could be trained with sufficient accuracy.

3.2 Hardware and Software Environment

Training the models is very resource-intensive compared to their mere use, so I outsourced the training in order to obtain results quickly.

Hardware setup:
The models were trained on the Kaggle platform, which provides a sufficiently powerful GPU that was indispensable for the training. The later tests were first run on my local system architecture and subsequently on a single-board computer, in this case the Raspberry Pi 5.

Software environment:

The system was implemented using the programming language Python [0]. The `ultralytics` package was used for the detector and the segmentation model. Image processing operations, such as extracting crops, were implemented with OpenCV [0]. The annotation and management of the segmentation dataset was carried out on the Roboflow platform [**roboflow2025**], from which the dataset was exported directly in YOLO format for training. For optimal inference speed, the models were exported to the TensorFlow Lite [0] format, as these are specifically optimised for ARM [0] architectures, which are used by our single-board computer.

Chapter 4

Methodology and Development

This chapter describes the conceptual development and technical implementation of the detection system presented in this thesis. It describes the path from the problem statement to the solution. In doing so, it examines in more detail the design decisions and the architecture of the individual components of the pipeline.

4.1 Overall Architecture and Data Flow

To reliably judge whether a cat is carrying a prey animal in its mouth, neither pure object detection nor pure image segmentation is sufficient. Since we only have a limited dataset of cats with prey animals available, it is essential to divide this task into several stages [**baumann2020smartcatflap**]. This results in a pipeline consisting of object detection, object tracking, image segmentation and temporal state modelling. The final pipeline looks as follows:

1. A frame is captured by the camera.
2. The YOLO object detection analyses the frame.
   (a) If no cat was found, the pipeline ends here.
   (b) If at least one cat is found, each cat face is extracted from the frame.
3. The ByteTrack algorithm assigns an identity to the cat faces or creates a new one if the cat has only just entered the frame.
4. The YOLO segmentation model examines each crop individually and returns for each cat a confidence score that it is carrying prey.
5. The state machine aggregates the results of the image segmentation over time and then makes a final decision about which state (e.g. “cat with prey”) the tracked objects are in.
   (a) If a cat is carrying prey in its mouth, the flap is closed.
   (b) If all cats have been recognised as clean over the entire time window, the flap is opened.

4.2 Preprocessing for Infrared Night Vision

Since cats do not only bring their prey home during the day under optimal lighting, a central requirement for the system is that it also works at night and in poor lighting. This brings a further difficulty: night vision cameras switch to infrared in the dark, which produces greyscale images.

So that the neural network does not learn the colour information, all images are converted to greyscale images before training and inference. Otherwise, neither object detection nor image segmentation would work reliably at night.

4.3 Training and Adaptation of the Object Detector (YOLO26n)

For object detection, the YOLO26 architecture in the `yolo26n` variant is used. It is the smallest one and offers high detection accuracy while providing the inference speed necessary for single-board computers.

Standard YOLO26 models are not specifically tuned to the detection of cat faces. Therefore, fine-tuning was carried out for exactly this problem [0]. The basis for this is the Kaggle dataset (Cat Dataset by Crawford [0]), since it already contains annotations for the face. The dataset contains the coordinates for eyes, ears and mouth. To make these usable for YOLO, they have to be converted into a bounding box format. For this purpose, an algorithm was developed that determines the width of the box from the ears and the height from the ears and the mouth.

The fine-tuning of the YOLO26n model was carried out over 100 epochs at an image resolution of 640x640 pixels and with the standard YOLO data augmentation [0]. To avoid overfitting of the model, the training is stopped if no improvement occurs in the last ten epochs [0].

4.4 Automated Generation of the Segmentation Dataset

Unfortunately, there was no usable dataset for the segmentation of whether the cat is carrying prey in its mouth. Two kinds of images are required for the training: cats with and cats without prey. The cats without prey can again be taken from the Kaggle dataset (Cat Dataset by Crawford [0]). Cats with prey, on the other hand, are extremely difficult to obtain; they are either of poor quality or protected by copyright.

I therefore had the images of cats with prey generated by Gemini [0]. Using the official Gemini Application Programming Interface (API), I developed an algorithm that, taking the usage limit into account, automatically generates images of cats with prey in different poses and at different distances.

To prepare the dataset for fine-tuning the YOLO26n segmentation model, the previously trained cat face detector is used. It detects the cat faces in the images without prey and crops them in exactly the same way as later during inference. These crops are added to the dataset as negative examples without annotation. The images with prey were provided with segmentation masks of the prey animals using the AI-assisted annotation of Roboflow [**roboflow2025**] and subsequently checked manually.

The segmentation model thus always has the same input conditions: it always re-

ceives the enlarged crop around the cat face without much background. This forms the perfect training basis, since during later inference the segmentation receives exactly the same input images.

4.5 Crop Extraction

Since the input of the segmentation model is the output of the detector, both must receive the required information. While the detector works best when it is trained only on the cat face, it also needs the information about what is located below the mouth of the cat face.

Therefore, the bounding box of the detector is modified before the segmentation model receives it. The cat face is extended downwards by twice the face height and sideways by 80 % of the face width on each side to make sure that the prey is in the image.

The detector expects a 3-channel image, so the greyscale image is duplicated three times (`cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)`). The segmentation model, by contrast, was trained directly on single-channel greyscale images and receives the crop as a single-channel image. Beforehand, the crop is padded with black borders to form a square.

4.6 Training of the Segmentation Model (YOLO Segmentation)

As in the case of the detector, a YOLO model is used as the segmentation model. Since we are already in the Ultralytics environment and modern YOLO versions provide highly optimised networks for segmentation tasks, this decision was an obvious one.

The model receives an image of 640x640 pixels and returns a segmentation mask and a confidence score for each detected prey animal. Because the model's dataset is relatively small, strong emphasis is placed on data augmentation (random rotation of the image by up to 10°, horizontal flipping, variation of brightness and saturation). Mosaic augmentation, in which four images are combined into one, is deactivated so that small prey animals are not lost. These augmentation steps make the model more robust against unfamiliar poses of the cats.

4.7 Object Tracking and Development of the State Model

When a cat approaches the flap, the quality of the images changes considerably due to motion blur or unfavourable angles. As a result, the outputs of the segmentation model fluctuate. If, for example, the prey is briefly occluded, this would mean that the cat is allowed into the house.

To mitigate this problem, the object detection was extended with the ByteTrack algorithm. As a result, a cat receives a unique object ID when it first appears in the frame, and this ID remains the same for that cat in the following frames. Based on the uniqueness of each cat, a state machine could now be developed. This class `StateMachine` manages a `TrackState` object for each detected cat (`track_id`). The model knows four states:

- `NO_CAT`: There is no relevant object in the field of view.
- `GATHERING_HISTORY`: A cat has been detected, but there are not yet enough frames for a decision.
- `CAT_NO_PREY`: A cat has been detected but is not carrying prey.
- `CAT_WITH_PREY`: A cat has been detected and is carrying prey with sufficient certainty.

So that the system's decision is not made on the basis of the most recent frame alone, the `TrackState` object stores a history of the confidence scores of the segmentation model for the last 20 frames. The current state is recalculated for every frame, simply by computing the moving average of the history. If the threshold of 0.3 is reached or exceeded, the cat is marked as `CAT_WITH_PREY`. If the value falls below it, the cat is marked as `CAT_NO_PREY`, provided that 20 values have already been collected.

The threshold of 0.3 was chosen on the basis of the parameter series in Section 6.4.1. Since the average over 20 frames already smooths out individual outliers considerably, the threshold can be set low without the cat having to stay outside unjustly. However, this value is not set in stone. If it were more important to the end user that the cat is allowed in when in doubt, the value would be set higher, for example at 0.5.

The decision about the flap results from the states of all cats. If at least one cat is carrying prey, the flap is locked immediately. It is only unlocked when all cats in the frame have a complete history and are considered clean. Only cats that are visible in the current frame are taken into account. If the tracker assigns a new identity to a cat, the short-lived track left behind therefore does not block the release. A cat with prey, on the other hand, maintains the lock until it is removed from memory. So that the flap does not constantly open and close, a renewed release is permitted at the earliest five seconds after the last change.

To prevent memory leaks in the state machine, cats that have left the field of view or that have not been found for more than 30 frames (`max_missed_frames`) are removed from memory.

Chapter 5

Design and Implementation

Having dealt with the methodology in the previous chapter, this chapter addresses the concrete realisation of the system. It explains how the architecture is structured and how the individual software components interlock. In addition, the data structures used are discussed.

5.1 Software Architecture and Core Components

The implementation was carried out in the Python programming language and adheres strictly to the rules of object-oriented programming so that the system remains maintainable and extensible. The core system was divided into three main scripts: detection models (`models.py`), state machine (`state_machine.py`) and orchestration (`processor.py`).

5.1.1 The Module `models.py`: The `CatFlapPipeline`

The class `CatFlapPipeline` manages the detector and the segmentation model and simplifies their use. When the object is initialised, both models are loaded. In doing so, the paths (absolute or relative) to the models are completed.

The method `run_detector` converts the frame to greyscale. The object detection is then called; calling the method `track` with the parameter `tracker='bytetrack.yaml'` activates the ByteTrack tracker integrated into Ultralytics, which provides us with the bounding boxes together with object IDs. The parameter `persist=True` ensures that the tracks are retained between calls.

The method `run_prey_detector` cuts out the crop as described in Section 4.5. The following code excerpt shows how this cropping is implemented and what effect it has on the cat face. By extending the crop downwards, we ensure that the model receives enough information about possible prey in the cat's mouth.

Listing 5.1: Asymmetric padding for the crop

```
x1, y1, x2, y2 = map(int, box)
# Add padding (heavy on bottom for dangling prey)
face_w = x2 - x1
face_h = y2 - y1
pad_w = int(face_w * 0.8)
```

```
pad_top = 0
pad_bottom = int(face_h * 2.0)
x1 -= pad_w
y1 -= pad_top
x2 += pad_w
y2 += pad_bottom
```

Subsequently, the crop of the cat face is processed by the segmentation model, which then returns a confidence score for each prey animal found. The highest of these is passed to the state machine. If the model finds no prey, the value is 0.

5.1.2 The Module `state_machine.py`: Temporal Aggregation

The state machine is represented by the class `StateMachine`. The status of an individual cat is stored in the class `TrackState`, which stores the history of the confidence scores in a `collections.deque` with a fixed maximum length (`history_length = 20`). This data structure ensures that, when the capacity limit is reached, the oldest value is removed from the queue ("First-In-First-Out"). This completely spares us manual memory management.

The following code excerpt shows the core logic of the `update` method. For each new frame, it calculates the moving average (`numpy.mean`) and then switches the state accordingly.

Listing 5.2: State determination using a moving average

```
def update(self, prey_confidence, frame_idx):
    self.confidence_history.append(prey_confidence)
    self.last_seen = frame_idx

    # Calculate moving average
    avg_confidence = np.mean(self.confidence_history)

    # Determine new state based on aggregated confidence
    if avg_confidence >= self.threshold:
        self.current_state = State.CAT_WITH_PREY
    elif len(self.confidence_history) < self.confidence_history.maxlen:
        self.current_state = State.GATHERING_HISTORY
    else:
        self.current_state = State.CAT_NO_PREY
```

The class `StateMachine` manages the various `TrackState` objects in a dictionary (`track_id` → `TrackState`). At the start of each call, the method `process_global_state` iterates over this dictionary and removes the objects that the tracker has not been able to match for more than 30 frames, in order to avoid memory leaks.

As soon as one or more cats are detected with prey in their mouth, the `StateMachine` object emits an event and reports that cats are not allowed to enter. Otherwise, i.e. if all cats are clean, the event reports that entry is allowed. The lock is reported immediately, whereas the release is only reported once all cats visible in the current frame have a complete history and at least five seconds have passed since the last change.

5.1.3 The Orchestration (`processor.py`)

The actual processing of the frames takes place in the script `processor.py`. The class `CatFlapProcessor` manages a `StateMachine` object, counts how many frames per second can be processed, and offers the method `process_frame`. This is the heart of the class; it receives a video frame as a parameter, after which it iterates over all detected cats, performing the segmentation for each object. In the last step, the state machine is then updated and the visualisations (bounding boxes and state) are drawn onto the frame.

How difficult it is to collect high-quality data of cats with prey was already mentioned in Chapter 3. To improve the segmentation model in a targeted way, a mechanism was implemented that saves frames for which the model was uncertain (confidence score between 0.3 and 0.6). This makes it possible to specifically improve the scenarios that the model finds most difficult. This feature is activated with the command-line parameter `-save_uncertain`. To save storage space, after the mechanism has been triggered, saving for the same `track_id` is locked for 30 frames.

5.2 The Connection to the Hardware (`catflap.py`)

The class `Catflap` contained in the script `catflap.py` provides the hardware environment. Through polymorphism, different variants can be created. The method `capture_continuous` returns the next frame from the camera. In addition, the class offers methods for closing (`lock_in` / `lock_out`) and opening (`unlock_in` / `unlock_out`) the flap.

- `lock_in`: Cat is not allowed to enter the house.
- `lock_out`: Cat is not allowed to leave the house.
- `unlock_in` and `unlock_out`: Remove the respective lock.

The subclass `SimulationCatflap` is used for simulation; here a video supplies the video frames, and locking and unlocking the flap is simulated by an output to the console. For real operation there are the classes `ServoCatflap`, which locks by means of a servo motor, and `SureCatflap`, which can be used as an interface to an existing SureFlap cat flap [0].

5.3 Execution Environments

Depending on the purpose, the system can be run in different environments, each of which has its own scripts.

5.3.1 The Local Test Environment (`main.py`)

The script `main.py` was developed to be able to quickly validate newly trained models in the local environment (e.g. a Windows laptop). The script reads in a video file. Each video frame then passes through the `process_frame` method of the class `CatFlapProcessor`. Subsequently, the visualisation is drawn onto the video and played back.

5.3.2 The Test Environment on the Raspberry Pi (`pi_main.py`)

To be able to sensibly test the interplay of all components on the target device, the script `pi_main.py` was developed. This makes it possible to test each part, that is, the detector, segmentation model, servo and camera, in isolation. In addition, this part is important for testing how many frames per second the Raspberry Pi 5 can process. Otherwise, the script works the same as `main.py`.

5.3.3 The Real World (`pi_server.py`)

For real operation, the previously mentioned scripts are not sufficient. So that the flap can be manually locked / unlocked during operation and the current camera recordings can be viewed, these functions are provided as HTTP endpoints in the script `pi_server.py` with the help of the `flask` package.

Here, too, a `CatFlapProcessor` object is created and managed, which continuously receives the individual frames from the camera. If the global access state of the `StateMachine` object contained within it changes, an event is emitted, as described in Section 5.1.2. If all cats are clean, the flap is opened; otherwise it is locked. Once the cat flap has been unlocked, a timer starts that closes the flap again after 30 seconds.

Chapter 6

Tests and Analyses

This chapter evaluates the performance of the entire system. The assessment is based on AI-generated video data that was not presented to the system during the training phase. The focus is on the question of whether the improvements through temporal aggregation and the two-stage pipeline are more reliable than using object detection and segmentation alone. It is also of interest in which scenarios these additions have a clear influence on the system.

6.1 Model Performance (Training & Validation)

Since the YOLO models have a significant influence on the reliability of the system, we first take a look at their isolated performance.

This section mainly addresses the training metrics. Of particular interest are the loss functions, which indicate how many errors occurred on the respective dataset (training or validation). The information from the confusion matrices provides insight into how many objects were detected, falsely detected or invented by the model.

6.1.1 Performance of Object Detection

The training metrics (Figure 6.1) for the cat face detector show a continuously decreasing training loss and a high precision in validation over the course of 100 epochs.

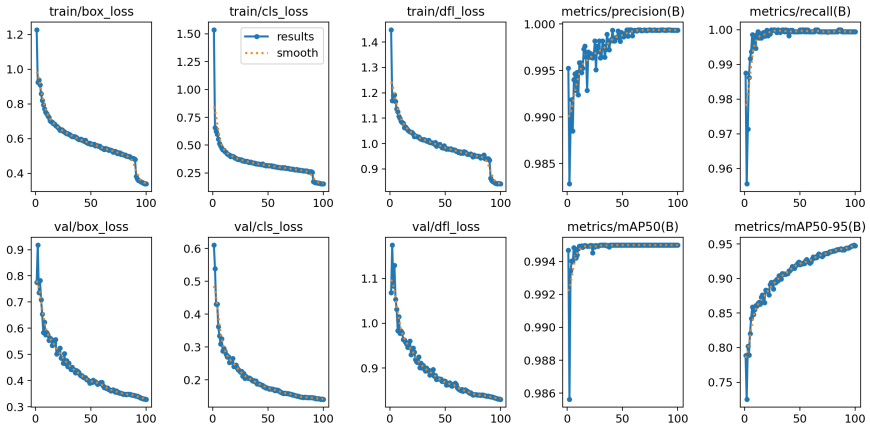


Figure 6.1: Training progress and metrics of the YOLO object detector.

The confusion matrix of the object detection (Figure 6.2) illustrates the precision of the localisation. It shows that 1,676 cat faces were correctly detected during validation. However, the model detected two cat faces where there was none, and it missed one completely. The matrix thus proves that the detector worked almost without errors on the validation data.

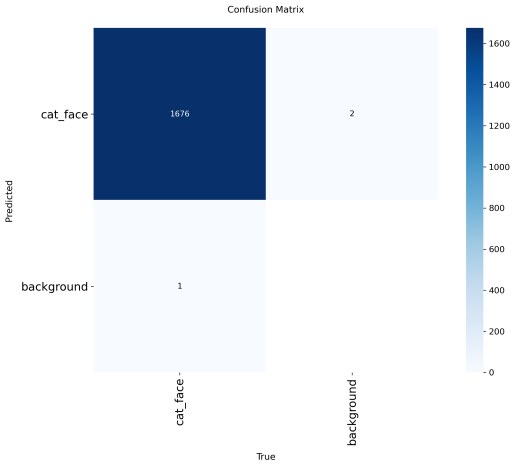


Figure 6.2: Confusion matrix (detector).

An important metric, which is also based on the validation data, is the precision-recall curve; it indicates how confident the model is, through the ratio between correctly detected objects and false alarms [0]. In this case, the curve shows that the model is extremely confident and does not have to guess when it sees a cat face.

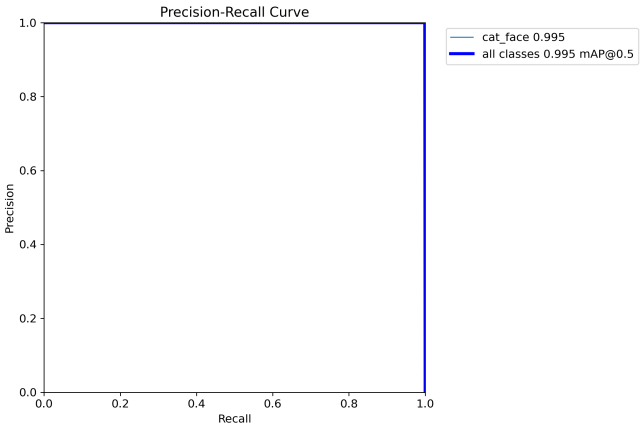


Figure 6.3: Precision-recall curve.

The F1 curve (Figure 6.4) shows how the performance of the model changes when the confidence threshold [0] is varied. This allows us to determine the optimal threshold for the real operation of the model. Since the curve stays flat up to ≈ 0.85, i.e. no cat is missed here, it drops sharply above 0.85. This shows that beyond this threshold, real cats are also easily missed. In practice, a confidence threshold of ≈ 0.75 yields a good balance between "cat is not detected" and "a cat is detected where there is none". In operation, the detector is nevertheless run with a low threshold of 0.1, since ByteTrack also uses detections with a low confidence score for the association [0].

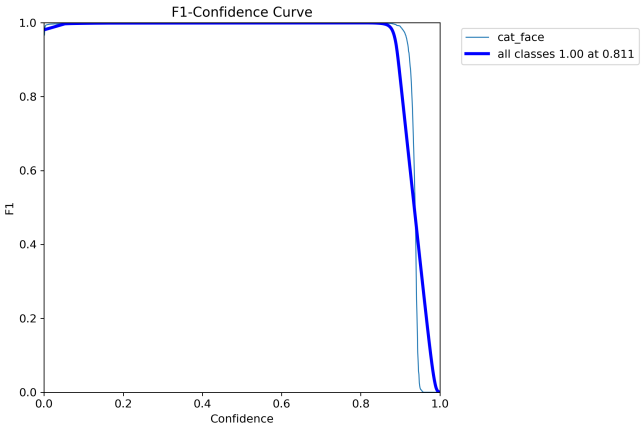


Figure 6.4: F1 curve.

Figure 6.5 gives an impression of the detection performance of the model when it is applied to the validation data.

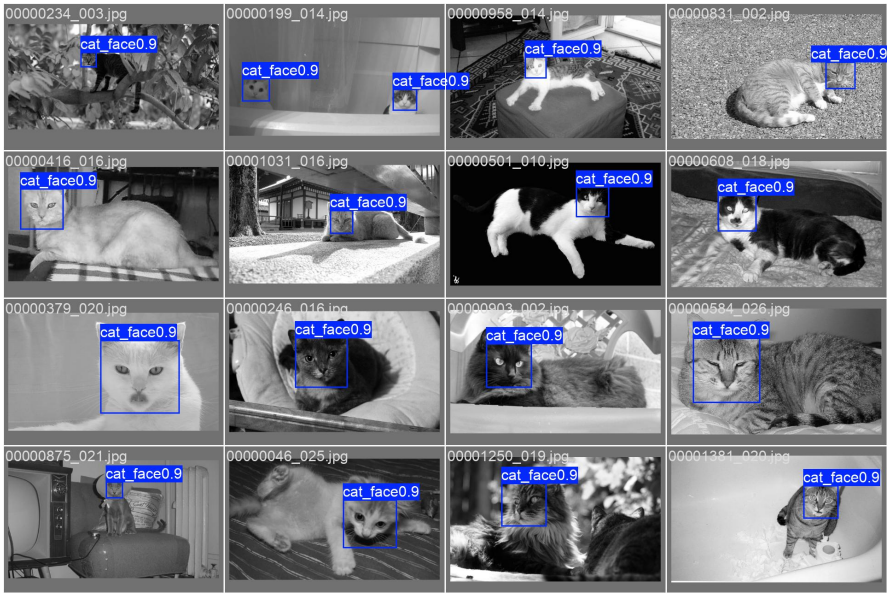


Figure 6.5: Predictions of the detector on the validation dataset.

6.1.2 Performance of Image Segmentation

The following training metrics of the prey segmentation model show that the model has successfully learned its task. Looking at the loss functions, it becomes apparent that the model makes fewer errors over the epochs in both training and validation; from this we can conclude that the segmentation model learns without overfitting too much to the training dataset.

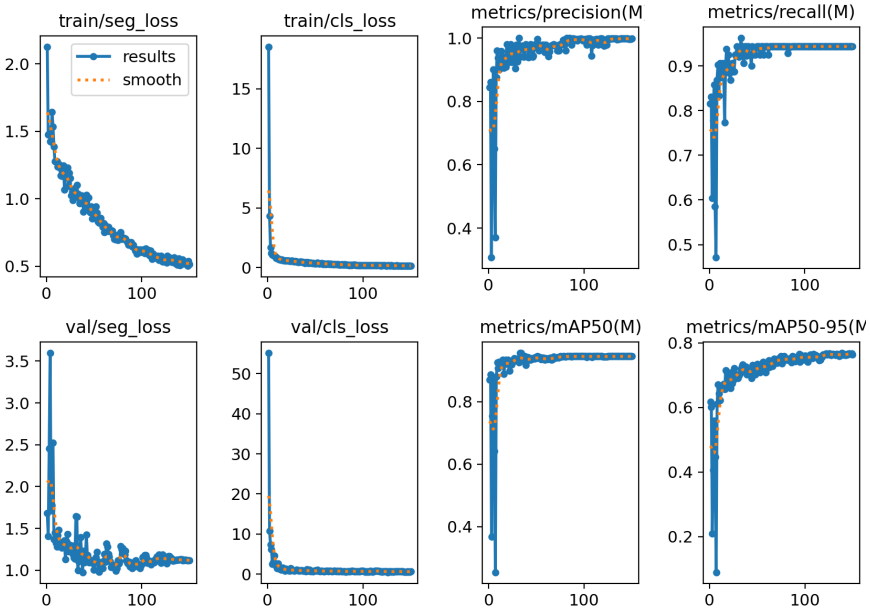


Figure 6.6: Training progress and metrics of the YOLO segmentation model.

The precision-recall curve of the segmentation masks (Figure 6.7) shows that the model produces not a single false detection up to a recall of about 94 %. On the validation dataset, it achieves an mAP50 of 0.945.

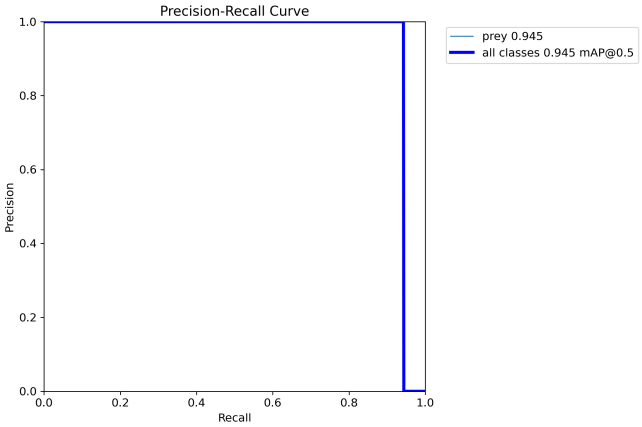


Figure 6.7: Precision-recall curve of the segmentation masks.

The F1 curve (Figure 6.8) reaches its maximum of 0.97 at a confidence threshold of 0.728. Between about 0.1 and 0.85 it remains nearly constant, so in this range the model is hardly sensitive to the choice of threshold.

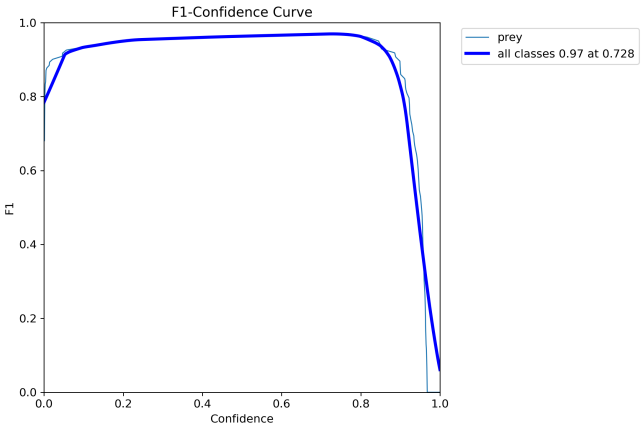


Figure 6.8: F1 curve of the segmentation model.

Figure 6.9 makes it easy to see how well the model handles unknown data.

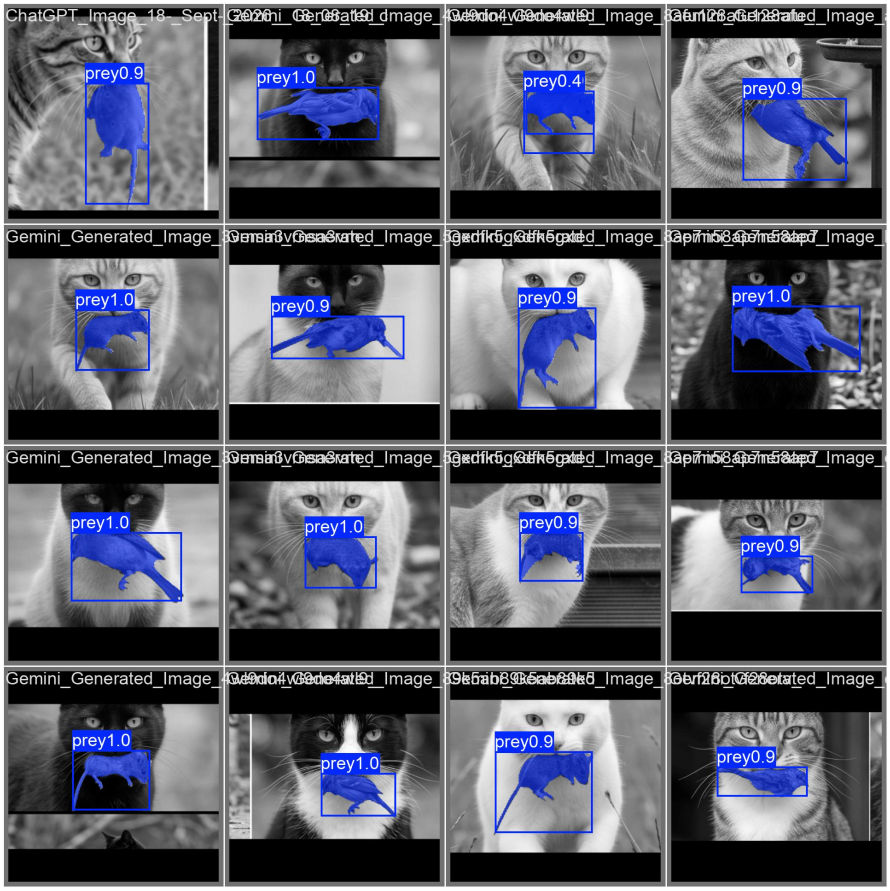


Figure 6.9: Example predictions of the segmentation model on the validation dataset.

On paper alone, these models would make almost error-free predictions. The decisive

question, however, is how the system behaves in practical use and which advantages the pipeline offers over pure detection and segmentation.

6.2 Evaluation Methodology and Dataset

For the evaluation of the overall system, a dedicated test dataset of AI-generated video sequences is created. This section describes the structure of the dataset, the annotation of the reference data and the metrics used.

6.2.1 Test Dataset

The test basis consists of ten video pairs generated with the video model Veo via Gemini [0]. Each video shows the perspective of a fixed camera mounted directly above the cat flap, towards which one or two cats are moving. A video pair consists of a sequence in which a cat carries a prey animal in its mouth, and a counterpart with an identical scene and, as far as possible, an identical movement sequence, in which no cat carries prey. The two videos of a pair therefore differ solely in the presence of the prey.

The video pairs cover different fur colours, prey animals (mouse, bird, rat and vole), lighting conditions (day, dusk and infrared night vision) as well as scenes with two cats. The videos have a length of ten seconds at 24 frames per second. Since the system on the Raspberry Pi 5 processes around 8 frames per second, the videos are thinned out to 8 frames per second for the evaluation. As a result, the measured decision times correspond to real operation.

6.2.2 Annotation of the Reference Data

A reference annotation is created for each video. For each cat, it specifies the time intervals in which the cat is visible in the image, in which its face is recognisable and in which it is carrying prey. From these intervals, the correct state of the cat flap results for each frame: if at least one cat with prey is in the image, the flap must be locked; otherwise it may be opened.

6.2.3 Definition of the System Metrics

After all video sequences have passed through the pipeline, the metrics for the entire system can be computed. These are derived from the confusion matrix:

- **True Positive (TP):** A cat with prey is not allowed into the house (correct locking).
- **False Negative (FN):** A cat with prey is allowed into the house (prey wrongly gets in).
- **True Negative (TN):** A cat without prey is allowed into the house (correct passage).
- **False Positive (FP):** A cat without prey is not allowed into the house (false alarm).

In the context of a cat flap, the cat has the highest priority from the user's point of view. A false positive (false alarm) means that the pet stands in front of a closed door for

no reason. In order not to destroy the animal's trust in the flap, the evaluation focuses primarily on avoiding such false alarms (high precision). An occasional false negative (a mouse gets into the house) is deliberately accepted in exchange for stress-free use of the flap by the cat.

In contrast to an evaluation at video level, the frame-wise evaluation also captures brief wrong decisions. For each frame with a visible cat, the state of the flap is compared with the reference annotation. This yields the following indicators:

- *Safety error*: Proportion of frames in which the flap is open although a cat with prey is visible.
- *Comfort error*: Proportion of frames in which the flap is locked although only clean cats are visible. Counting only starts after a decision time of three seconds from the detection of the cat face, since the state machine deliberately locks at the beginning.
- *Precision*, *recall* and *F1 score* of the decision "lock flap" over all frames with a visible cat. Frames with clean cats within the decision time are not taken into account.
- *Prey let through*: Number of videos with prey in which the flap was opened at least once.
- *Decision time*: Time from the detection of the cat face until the first correct decision, i.e. until locking in the case of a cat with prey or until release in the case of a clean cat.
- *Missed cats*: Proportion of frames with a recognisable cat face in which the system detects no cat.

The evaluation takes place in two steps. First, the detector and the segmentation model are run once per video and configuration, and the results of each frame are logged. Subsequently, the state machine is replicated on these logs. This makes it possible to compare different parameters of the state machine without repeating inference.

6.3 Evaluation Results

The overall system is evaluated in its final configuration: the face detector and the segmentation model work on greyscale images, and the state machine uses a time window of 20 frames and a threshold of 0.3. The derivation of these parameters is described in Section 6.4.1. Table 6.1 summarises the results on the 20 videos of the test dataset.

Metric	Videos with prey	Videos without prey
Safety error	1.0 %	–
Comfort error	–	3.3 %
Prey let through	2 of 10	–
Clean cats locked out	–	0 of 10
Decision time	0.13 s	0.60 s

Table 6.1: Results of the overall system on the test dataset.

Over all frames with a visible cat, the decision "lock flap" achieves a precision of 0.984, a recall of 0.990 and an F1 score of 0.987. In 4.3 % of the frames with a recognisable cat face, the system detects no cat. Broken down by lighting conditions, the safety error is 0.8 % for daytime recordings and 1.2 % for infrared recordings, and the comfort error is 5.6 % and 0.0 % respectively.

No clean cat is locked out permanently. In two of the ten videos with prey, the flap is briefly opened for four frames each (0.5 seconds). The analysis of these videos shows two causes:

- At the beginning, the cat is still far from the camera, so the prey in the crop is too small to be detected by the segmentation model. If the average after 20 frames is still below the threshold, the cat is briefly released until the prey is detected.
- At the end of the video, the cat walks underneath the camera. The segmentation model returns no prey for these frames, and after the time window has elapsed, the cat is considered clean.

In the scenes with two cats, the tracker (ByteTrack) sometimes assigns the same cat a new identity several times. The resulting short-lived tracks do not block the release, since only cats that are visible in the current frame are taken into account. The release is thereby delayed in these scenes by up to 2.6 seconds, but remains within the decision time.

6.4 Ablation Studies

In the following ablation studies, one component of the system is removed or replaced in each case, while all other components remain unchanged. All studies use the test dataset and the metrics from Section 6.2.

6.4.1 State Machine

To validate the effectiveness and absolute necessity of the temporal aggregation (state machine), an ablation study was carried out. Note: An initial lock is intended: even if the cat is not carrying prey, it is denied access until it proves that it is clean. Comfort errors are therefore only counted after the decision time of three seconds. In this study, the exact same system was applied twice to all 20 test videos:

1. **Full system:** With the state machine active and a moving average over 20 frames (`history_length=20`).
2. **Ablation:** Since the state machine was deactivated, the decision about locking the flap was based exclusively on the result of the current frame (`history_length=1`).

Table 6.2 compares both configurations. For the decision based on the current frame, a threshold of 0.5 is used; for the full system, the threshold of 0.3 from the subsequent parameter sweep.

Metric	Without state machine	With state machine
Safety error	17.0 %	1.0 %
Comfort error	19.7 %	3.3 %
Precision	0.896	0.984
Recall	0.830	0.990
F1 score	0.862	0.987
Prey let through	10 of 10	2 of 10
Clean cats locked out	0 of 10	0 of 10

Table 6.2: Two-stage system without state machine (time window of one frame, threshold 0.5) and with state machine (time window of 20 frames, threshold 0.3).

Without the state machine, the flap is open in 17.0 % of the frames with prey, and in each of the ten videos with prey the flap is released at least once. At the same time, the flap is locked in 19.7 % of the frames with clean cats. Individual wrong decisions of the segmentation model directly affect the state of the flap without temporal aggregation. With the state machine, the safety error drops to 1.0 % and the comfort error to 3.3 %.

In addition, a parameter sweep over the length of the time window (1, 3, 5, 8, 10, 15 and 20 frames) and the threshold (0.3 to 0.8) is evaluated. It shows the trade-off between safety errors, comfort errors and decision time and serves to determine the final parameters. Table 6.3 contains the results of all combinations. The proportion of missed cats is 4.3 % in all combinations, since it depends only on the detector. In no combination is a clean cat locked out permanently.

Window	Threshold	Safety err.	Comfort err.	F1	Prey let through	Decision time [s] Prey	Decision time [s] clean
1	0.3	11.6 %	19.2 %	0.894	9/10	0.03	0.00
1	0.4	12.5 %	19.2 %	0.889	10/10	0.03	0.00
1	0.5	17.0 %	19.7 %	0.862	10/10	0.46	0.00
1	0.6	17.2 %	20.3 %	0.859	10/10	0.46	0.00
1	0.7	18.5 %	20.3 %	0.852	10/10	0.46	0.00
1	0.8	23.8 %	20.3 %	0.819	10/10	0.58	0.00
3	0.3	10.5 %	20.0 %	0.898	9/10	0.03	0.00
3	0.4	11.2 %	20.3 %	0.894	9/10	0.03	0.00
3	0.5	16.2 %	20.5 %	0.865	9/10	0.46	0.00
3	0.6	20.2 %	12.3 %	0.859	10/10	0.46	0.00
3	0.7	25.8 %	10.5 %	0.828	10/10	0.46	0.00
3	0.8	31.1 %	10.5 %	0.792	10/10	0.06	0.00
5	0.3	7.1 %	20.8 %	0.915	9/10	0.03	0.00
5	0.4	7.8 %	20.5 %	0.912	9/10	0.03	0.00
5	0.5	21.2 %	10.5 %	0.857	9/10	0.51	0.00
5	0.6	24.1 %	3.3 %	0.855	10/10	0.54	0.00
5	0.7	25.6 %	3.3 %	0.845	10/10	0.61	0.00
5	0.8	30.8 %	3.3 %	0.811	10/10	0.43	0.00
8	0.3	5.4 %	18.7 %	0.929	8/10	0.04	0.11
8	0.4	9.6 %	12.6 %	0.920	9/10	0.05	0.11
8	0.5	16.4 %	3.3 %	0.903	9/10	0.55	0.11
8	0.6	19.5 %	3.3 %	0.884	9/10	0.57	0.11
8	0.7	23.1 %	3.3 %	0.861	10/10	0.65	0.11
8	0.8	33.6 %	3.3 %	0.790	10/10	0.36	0.11
10	0.3	4.0 %	12.6 %	0.950	6/10	0.05	0.14
10	0.4	5.9 %	3.3 %	0.962	8/10	0.06	0.14
10	0.5	11.9 %	3.3 %	0.929	9/10	0.54	0.14
10	0.6	16.4 %	3.3 %	0.903	9/10	0.64	0.14
10	0.7	19.1 %	3.3 %	0.886	9/10	0.73	0.14
10	0.8	36.2 %	3.3 %	0.771	10/10	0.39	0.14
15	0.3	1.8 %	3.3 %	0.983	4/10	0.08	0.20
15	0.4	4.1 %	3.3 %	0.971	5/10	0.16	0.20
15	0.5	11.4 %	3.3 %	0.932	6/10	0.66	0.20
15	0.6	13.1 %	3.3 %	0.922	8/10	0.79	0.20
15	0.7	14.9 %	3.3 %	0.912	9/10	0.96	0.20
15	0.8	28.7 %	3.3 %	0.824	10/10	0.67	0.20
20	**0.3**	**1.0 %**	**3.3 %**	**0.987**	**2/10**	**0.13**	**0.60**
20	0.4	1.8 %	3.3 %	0.983	3/10	0.21	0.60
20	0.5	6.9 %	3.3 %	0.956	6/10	0.78	0.60
20	0.6	8.4 %	3.3 %	0.948	6/10	1.00	0.60
20	0.7	11.0 %	3.3 %	0.934	9/10	1.31	0.60
20	0.8	24.4 %	3.3 %	0.853	10/10	1.19	0.60

Table 6.3: Parameter sweep of the state machine over the length of the time window (in frames) and the threshold. The selected configuration is highlighted in bold. A time window of one frame corresponds to a system without a state machine.

The following relationships can be read from the parameter sweep:

- *Length of the time window*: At a threshold of 0.3, both the safety error (from 11.6 % to 1.0 %) and the number of videos in which prey is let through (from 9 to 2) decrease as the time window grows longer. Brief wrong decisions of the

segmentation model are averaged over more frames and less often lead to a release. This relationship cannot be observed at high thresholds. From a time window of ten frames onwards, the comfort error remains constant at 3.3 % for thresholds of 0.4 and above.

- *Threshold*: A low threshold locks the flap even at a small average proportion of prey. This reduces the safety error, but for time windows of three to ten frames the comfort error increases. For long time windows, the comfort error remains at 3.3 % even at a threshold of 0.3.
- *Decision time*: Longer time windows delay the release for clean cats, since the state machine first has to collect a sufficient history. At the same time, the time until locking for cats with prey generally increases with the threshold. At 20 frames, the mean decision time for clean cats is 0.60 seconds and is thus clearly below the decision time of three seconds.

The combination of a time window of 20 frames and a threshold of 0.3 achieves the lowest safety error (1.0 %), the highest F1 score (0.987) and the lowest number of videos with prey let through (2 of 10). The comfort error of 3.3 % is the lowest value of the entire parameter sweep, and no clean cat is locked out. At 8 frames per second, the time window corresponds to a history of 2.5 seconds. This configuration is therefore set as the final parametrisation of the state machine and used in all following ablation studies.

6.4.2 Detection and Segmentation versus Detection

To validate the architecture of our two-stage chain (detector + segmentation model), a single-stage object detector (YOLO26) was trained for this study. It has two classes: “cat with prey” and “cat without prey”. Both classes contain the same amount of data.

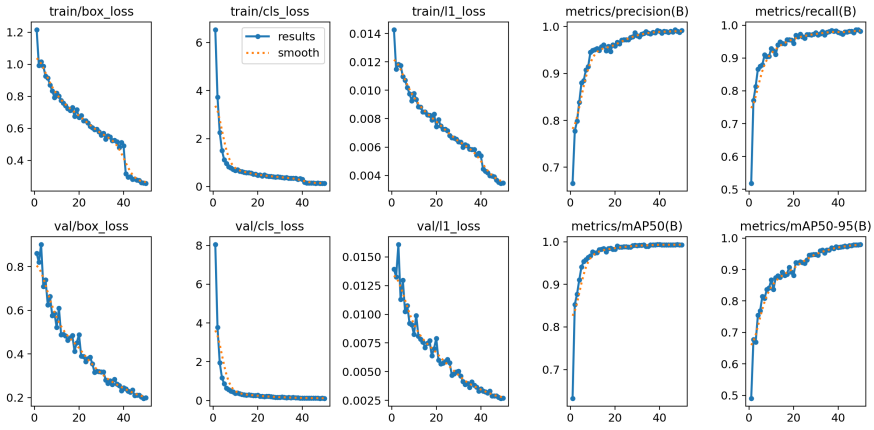


Figure 6.10: Training metrics of the single-stage object detector.

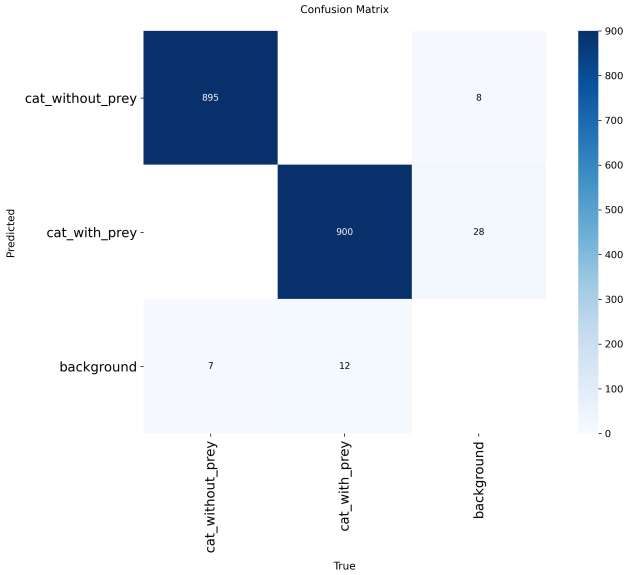


Figure 6.11: Confusion matrix of the single-stage object detector on the validation data.

To compare the models in isolation, the state machine was deactivated again (`history_length=1`). We evaluated how many safety and comfort errors both systems made, as well as how many cats were missed in the video overall. In addition, both systems are evaluated with the state machine in the final configuration (time window of 20 frames, threshold 0.3). Table 6.4 shows the results of both variants.

	Without state machine		With state machine	
Metric	Two-stage	Single-stage	Two-stage	Single-stage
Safety error	17.0 %	9.9 %	1.0 %	10.6 %
Comfort error	19.7 %	61.5 %	3.3 %	29.7 %
Precision	0.896	0.750	0.984	0.860
Recall	0.830	0.901	0.990	0.894
F1 score	0.862	0.819	0.987	0.877
Missed cats	4.3 %	6.5 %	4.3 %	6.5 %
Prey let through	10 of 10	8 of 10	2 of 10	2 of 10
Decision time (clean)	0.00 s	2.54 s	0.60 s	3.02 s

Table 6.4: Comparison of the two-stage system (detection and segmentation) with the single-stage object detector, without state machine (time window of one frame, threshold 0.5) and with state machine (time window of 20 frames, threshold 0.3).

At 6.5 %, the single-stage system misses more cats than the face detector of the two-stage system (4.3 %). The clearest difference is in the comfort error: without the state machine, the flap is locked in 61.5 % of the frames with clean cats, and with the state machine in 29.7 %. The single-stage detector frequently assigns clean cats to the

class “cat with prey”. As a result, without the state machine it achieves a lower safety error than the two-stage system, but this comes at the expense of the clean cats. With the state machine, both systems let prey through in two of ten videos. However, the safety error of the single-stage system, at 10.6 %, is around ten times as high as that of the two-stage system (1.0 %). The F1 score of the two-stage system, at 0.987, is clearly above that of the single-stage system (0.877). In addition, the single-stage system needs 3.02 seconds on average until a clean cat is released and thus exceeds the decision time of three seconds.

6.4.3 Greyscale versus Colour Images

Since the camera switches to infrared night vision in darkness, the system works on greyscale images throughout. In this study, a system trained on greyscale images is compared with a system trained on colour images, broken down by daytime and nighttime recordings. Both segmentation models have the same architecture (YOLO26n-seg) and are trained with the same training data. The colour model receives the original colour images, the greyscale model the images converted to greyscale. The face detector and the state machine (time window of 20 frames, threshold 0.3) remain unchanged in both configurations. The daytime recordings comprise six video pairs, the infrared recordings four. Table 6.5 shows the results.

	Day		Infrared		Overall	
Metric	Grey	Colour	Grey	Colour	Grey	Colour
Safety error	0.8 %	4.4 %	1.2 %	5.6 %	1.0 %	4.9 %
Comfort error	5.6 %	0.0 %	0.0 %	0.0 %	3.3 %	0.0 %
Precision	0.973	1.000	1.000	1.000	0.984	1.000
Recall	0.992	0.956	0.988	0.944	0.990	0.951
F1 score	0.982	0.978	0.994	0.971	0.987	0.975
Prey let through	1 of 6	2 of 6	1 of 4	2 of 4	2 of 10	4 of 10
Decision time (prey)	0.19 s	0.94 s	0.03 s	0.75 s	0.13 s	0.86 s

Table 6.5: Comparison of the segmentation model for greyscale images with the segmentation model for colour images, broken down by daytime and infrared recordings.

For daytime recordings, both models achieve a similar F1 score (0.982 versus 0.978). The greyscale model has a lower safety error (0.8 % versus 4.4 %), the colour model a lower comfort error (0.0 % versus 5.6 %). For infrared recordings, no colour information is available. Here the greyscale model achieves a clearly lower safety error (1.2 % versus 5.6 %) and a higher F1 score (0.994 versus 0.971), while neither model shows a comfort error. In addition, the greyscale model locks the flap for cats with prey after 0.13 seconds on average, the colour model only after 0.86 seconds. Over the entire test dataset, the greyscale model achieves the higher F1 score and lets prey through in two instead of four videos.

Chapter 7

Discussion

The test results presented in Chapter 6 provide a clear picture of the system performance. In this chapter, these results are interpreted and critically assessed. In doing so, it is examined to what extent the research goals were achieved, where the limits of the system lie and which functionalities were decisive for its quality.

7.1 Answering the Central Research Question

The primary research question of this thesis was whether a pipeline consisting of object detection, image segmentation, object tracking and temporal state modelling enables a more robust detection of prey compared with single-frame methods.

The evaluation provides a clear answer to this. A single-frame approach is highly susceptible to errors when analysing video streams. Motion blur when the cat walks into the image, partial occlusions of the prey by the paws or rapid changes in lighting lead to severe inaccuracies in the segmentation. Without the temporal aggregation in the state machine, the access control would constantly switch between the states “locked” and “unlocked” (flickering).

With the help of the `StateMachine` in combination with ByteTrack, this problem is solved elegantly. The temporal aggregation over a history of frames produces a robust signal that effectively smooths out outliers. The ablation study in Section 6.4.1 shows that the system makes the correct decision even when individual frames are segmented incorrectly. The safety error drops from 17.0 % to 1.0 %, the comfort error from 19.7 % to 3.3 %. This proves that the state machine is an essential part in this use case and is imperative for reliability in real-world operation.

7.2 Assessment of the Architectural Approach

The decision to divide the problem into two separate neural networks (YOLO detector for the face and YOLO segmentation model for the prey) has proven to be extremely advantageous.

7.2.1 Strengths of the Pipeline Cascading

The usual approach in the field of computer vision would be to train a single object detection model to directly find the classes "cat" and "prey animal" in the overall image. In the case of a cat flap, however, this approach often fails. A mouse in a cat's mouth occupies only a vanishingly small fraction of the pixels in the camera's overall image and is heavily occluded by the mouth and the whiskers. Standard object detectors have immense difficulty in reliably placing a bounding box around such small, occluded objects [**tong2020small**].

The cascading approach chosen in this thesis elegantly circumvents this problem [**baumann2020smartcatflap**, **zhang2014partbased**]. The first detector only has to find the comparatively large and distinctive cat face – a trivial task for modern architectures such as YOLO26, which was reflected in the high detection stability. By subsequently cutting out (cropping) this region, the attention of the downstream segmentation model is focused exclusively on the relevant region (attention mechanism *by design*). The segmentation model does not have to search for a small mouse in the background of the garden, but merely analyses the pixel structure directly at the cat's mouth. The ablation study in Section 6.4.2 confirms this: the single-stage detector locks clean cats in 29.7 % of the frames, the two-stage system only in 3.3 %. In addition, the safety error of the single-stage detector is around ten times as high.

7.2.2 The Role of the Asymmetric Padding

Because the annotations of the dataset in the detector ("Cat Dataset" by Crawford [0]) completely cover the lower part of the mouth, it is essential for training and inference to extend the bounding box proportionally downwards.

7.3 Weaknesses, Limitations and Sources of Error

The analysis clearly shows the limitations of the current system. This gives rise to the potential for future improvements:

1. **Loss of depth information:** Because the camera records a 2D image, it is not possible to distinguish how far away an object below the cat's mouth is. It follows that the model cannot reliably separate the background from the prey.
2. **Extreme speed of movement:** When a cat runs through the flap in a panic, very little time remains to make a clear statement. At a runtime of approx. 8 FPS, the system has only a handful of frames left for analysis. Since the state machine collects 20 frames (2.5 seconds) before it releases a cat, the cat must be in the image for at least that long. In this scenario, the `StateMachine` cannot fully exploit its strength (the temporal smoothing) and the cat has to wait until the state machine makes a clear decision.
3. **Long distance:** If the cat is still far from the camera, the prey in the crop is too small to be detected by the segmentation model. In the evaluation, this led to a brief release in one video before the prey was detected. Here it is essential to collect data from real-world operation in which the model was uncertain, in order to be able to train for precisely these cases better.

4. **Loss of the cat's face:** If the cat walks directly underneath the camera or turns its head away, the segmentation model no longer returns any prey. After the time window has elapsed, the cat is then considered clean. One possible approach would be to keep the lock on a cat with prey in place until the cat has left the image.

7.4 Local Data Processing

One aim of this thesis was also that the system be designed in a resource-efficient manner to ensure that inference can run locally on a single-board computer. The decision was therefore made in favour of a lightweight architecture (YOLO26n) and optimised formats (TensorFlow Lite). The accuracy lost as a result is justified by the performance achieved (approx. 8 FPS).

This approach proves that a complex task can be solved on resource-limited hardware through an intelligently chosen architecture without the robustness being impaired too much. This avoids outsourcing computations to external data centres. This saves money and makes the solution more reliable, since no internet connection is necessary.

In conclusion, it can be said that the architecture largely fulfils its task. The remaining false alarms are not an architectural flaw, but show the limits of the limited segmentation dataset. With the help of the model integrated in Chapter 5 for saving images in which the model was uncertain, precisely these gaps in the dataset can be filled. In this way, the model can be continuously improved.

Chapter 8

Summary and Outlook

In this bachelor thesis, a robust access system for cats is developed that prevents prey animals (e.g. mice or birds) from entering the living space. By combining deep learning and temporal aggregation, a system was created that outperforms single-frame object detection by a wide margin.

8.1 Summary of the Findings

The theoretical assumptions could be successfully validated through the practical evaluation. In this use case, the classic monolithic object detector is far less accurate than the pipeline presented in this thesis, which consists of an object detector, an image segmentation and a temporal aggregation.

First, a YOLO detector detects the cat's face. To prevent the possible prey in the cat's mouth from being occluded, the crop of the face is extended downwards by twice the face height. In the second step, a YOLO segmentation model analyses whether there is prey in the cat's mouth. To compensate for misclassifications, a cat has to prove that it is clean. Only then may it enter the living space. For this purpose, the cat is tracked over several seconds using ByteTrack. The moving average of the last 20 frames (which corresponds to 2.5 seconds at 8 frames per second) then decides whether or not the cat is allowed into the house.

The evaluation with AI-generated video data showed the success of the chosen architectural approach. The system wrongly released the flap in only 1.0 % of the frames with prey and never permanently locked out a clean cat. The chosen threshold of 0.3 locks the flap even at slight signs of prey. Thanks to the smoothing over 20 frames, clean cats nevertheless do not have to wait unnecessarily. The system thus proves to be very practical and runs in a resource-efficient manner on local edge devices at an average of 8 frames per second.

The central research question could therefore be answered in the affirmative. A combination of object detection, object tracking and temporal smoothing is considerably more robust than conventional single-frame evaluation.

8.2 Outlook and Possibilities for Further Development

Although the system in its current form already reliably keeps prey animals out, practice shows that this approach also has weaknesses and limitations. This makes it possible to identify concrete starting points for future improvements.

8.2.1 Continuous Collection of Hard Negative Examples

The most promising method to increase system performance is to expand the dataset for cats with prey. To automate this goal, the application has a functionality that saves images for which the image segmentation was uncertain. Future work can thereby use this steadily growing dataset to iteratively retrain the YOLO segmentation model. Once the segmentation model can learn from these errors, the false alarm rate of the entire system will decrease considerably.

8.2.2 Loss of Depth Information

A further limitation is the loss of 3D information in the image. As a result, it can happen that objects in the background or even the cat's body are wrongly interpreted as "in the cat's mouth". One possible extension would be to recover this 3D information with the help of stereo cameras or time-of-flight sensors. This could filter out false alarms in which the supposed prey animal is located far behind the cat's mouth.

8.2.3 Hardware Acceleration

Currently, the system is executed exclusively on the CPU. Switching to dedicated hardware (e.g. Google Coral) would significantly increase the number of frames processed per second. As a result, we can collect more data on the cat in the same amount of time, which would improve the temporal smoothing. The flickering of the system would thereby be reduced even further.

In conclusion, it can be said that the pipeline developed in this thesis forms an extremely compact and robust foundation. It reliably solves the problem of cats bringing in unwanted prey and at the same time allows for exciting extensions and improvements.

References

Literature

[0] Martín Abadi et al. “Tensorflow: A system for large-scale machine learning”. In: *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 2016, pp. 265–283 (cit. on p. 6).

[0] Arm Limited. *Arm Architecture*. Verfügbar unter: https://www.arm.com/architecture. 2026 (cit. on p. 6).

[0] Alex Bewley et al. “Simple Online and Realtime Tracking”. In: *IEEE International Conference on Image Processing (ICIP)*. 2016, pp. 3464–3468 (cit. on p. 3).

[0] Daniel Bolya et al. “YOLACT: Real-Time Instance Segmentation”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 9157–9166 (cit. on p. 3).

[0] Gary Bradski. “The OpenCV Library”. *Dr. Dobb’s Journal of Software Tools* (2000) (cit. on p. 6).

[0] Crawford. *Cat Dataset*. Verfügbar unter: https://www.kaggle.com/datasets/crawford/cat-dataset. 2024 (cit. on pp. 5, 8, 30).

[0] Mark Everingham et al. “The PASCAL Visual Object Classes (VOC) Challenge”. *International Journal of Computer Vision* 88.2 (2010), pp. 303–338 (cit. on p. 16).

[0] Gemini Team et al. “Gemini: a family of highly capable multimodal models”. *arXiv preprint arXiv:2312.11805* (2023) (cit. on pp. 8, 21).

[0] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016 (cit. on p. 8).

[0] Wei Han et al. *Seq-NMS for Video Object Detection*. arXiv:1602.08465. 2016. arXiv: 1602.08465 (cit. on p. 3).

[0] David Harel. “Statecharts: A Visual Formalism for Complex Systems”. *Science of Computer Programming* 8.3 (1987), pp. 231–274 (cit. on p. 3).

[0] Kaiming He et al. “Mask R-CNN”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2961–2969 (cit. on p. 3).

[0] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Pearson Addison-Wesley, 2006 (cit. on p. 3).

[0] Licheng Jiao et al. “New Generation Deep Learning for Video Object Detection: A Survey”. *IEEE Transactions on Neural Networks and Learning Systems* (2022) (cit. on p. 1).

[0] Glenn Jocher et al. *Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models*. arXiv:2606.03748. 2026. arXiv: 2606.03748 (cit. on pp. 1–3).

[0] Rudolph Emil Kalman. “A new approach to linear filtering and prediction problems”. *Journal of basic Engineering* 82.1 (1960), pp. 35–45 (cit. on p. 3).

[0] Kai Kang et al. “T-CNN: Tubelets With Convolutional Neural Networks for Object Detection From Videos”. *IEEE Transactions on Circuits and Systems for Video Technology* 28.10 (2018), pp. 2896–2907 (cit. on p. 3).

[0] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 25. 2012 (cit. on p. 2).

[0] Harold W. Kuhn. “The Hungarian Method for the Assignment Problem”. *Naval Research Logistics Quarterly* 2.1–2 (1955), pp. 83–97 (cit. on p. 3).

[0] Yann LeCun et al. “Gradient-based learning applied to document recognition”. *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324 (cit. on pp. 2, 3).

[0] Tsung-Yi Lin et al. “Microsoft COCO: Common Objects in Context”. In: *Computer Vision – ECCV 2014*. Vol. 8693. Lecture Notes in Computer Science. Springer, 2014, pp. 740–755 (cit. on p. 16).

[0] Jonathan Long, Evan Shelhamer, and Trevor Darrell. “Fully Convolutional Networks for Semantic Segmentation”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 3431–3440 (cit. on p. 3).

[0] Wenhan Luo et al. “Multiple Object Tracking: A Literature Review”. *Artificial Intelligence* 293 (2021), p. 103448 (cit. on p. 2).

[0] George H. Mealy. “A Method for Synthesizing Sequential Circuits”. *The Bell System Technical Journal* 34.5 (1955), pp. 1045–1079 (cit. on p. 3).

[0] Shervin Minaee et al. “Image Segmentation Using Deep Learning: A Survey”. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.7 (2022), pp. 3523–3542 (cit. on p. 3).

[0] Edward F. Moore. “Gedanken-Experiments on Sequential Machines”. In: *Automata Studies*. Ed. by Claude E. Shannon and John McCarthy. Vol. 34. Annals of Mathematics Studies. Princeton University Press, 1956, pp. 129–153 (cit. on p. 3).

[0] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 779–788 (cit. on p. 2).

[0] Shaoqing Ren et al. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 28. 2015 (cit. on p. 2).

[0] Ranjan Sapkota et al. *YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection*. arXiv:2509.25164. 2025. arXiv: 25 09.25164 (cit. on p. 2).

[0] Sure Petcare. *Microchip Cat Flap*. Verfügbar unter: https://www.surepetcare.com /en-gb/pet-doors/microchip-cat-flap. 2026 (cit. on p. 13).

[0] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer Nature, 2022 (cit. on p. 2).

[0] Juan Terven, Diana-Margarita Córdova-Esparza, and Julio-Alejandro Romero-González. "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS". *Machine Learning and Knowledge Extraction* 5.4 (2023), pp. 1680–1716 (cit. on p. 2).

[0] Ultralytics. *Glossary: Confidence*. Verfügbar unter: https://www.ultralytics.com/de /glossary/confidence. 2026 (cit. on pp. 2, 17).

[0] Ultralytics. *YOLO Data Augmentation*. Verfügbar unter: https://docs.ultralytics.c om/guides/yolo-data-augmentation. 2024 (cit. on p. 8).

[0] Guido Van Rossum and Fred L Drake Jr. *Python tutorial*. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995 (cit. on p. 6).

[0] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. "Simple Online and Realtime Tracking with a Deep Association Metric". In: *IEEE International Conference on Image Processing (ICIP)*. 2017, pp. 3645–3649 (cit. on p. 3).

[0] Jason Yosinski et al. "How Transferable Are Features in Deep Neural Networks?" In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 27. 2014 (cit. on p. 8).

[0] Yifu Zhang et al. "ByteTrack: Multi-Object Tracking by Associating Every Detection Box". In: *Computer Vision – ECCV 2022*. Vol. 13682. Lecture Notes in Computer Science. Springer, 2022, pp. 1–21 (cit. on pp. 1, 3, 17).

[0] Zhengxia Zou et al. "Object Detection in 20 Years: A Survey". *Proceedings of the IEEE* 111.3 (2023), pp. 257–276 (cit. on p. 2).